Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Фундаментальные науки»

КАФЕДРА «Прикладная математика»

Лабораторная работа № 1
по дисциплине «Типы и структуры данных»

Тема Алгоритмы умножения матриц

Студент Михайлов А.К.

Группа ФН12-31Б

Преподаватели Волкова Л.Л.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Классический алгоритм умножения матриц 5

1.2 Метод Винограда умножения матриц 5

1.3 Оптимизированный метод Винограда умножения матриц 6

2 Конструкторская часть **7**

2.1 Классический алгоритм умножения матриц 7

2.2 Алгоритм Винограда умножения матриц 8

2.3 Оптимизированный алгоритм Винограда умножения матриц 9

3 Технологическая часть **10**

3.1 Реализация и выбор средств 10

3.2 Тестирование программы 10

4 Исследовательская часть **14**

4.1 Сравнение времени выполнения программы 14

4.2 Оценка трудоемкости алгоритмов 14

4.3 Вывод 16

ЗАКЛЮЧЕНИЕ **17**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **18**

ПРИЛОЖЕНИЕ А **19**

ВВЕДЕНИЕ

В данной лабораторной работе рассматриваются три алгоритма (**классический, Винограда, Винограда оптимизированный**), позволяющие выполнять умножение матриц произвольных размеров (в рамках допустимых математических условий).

Целью выполнения является реализация указанных алгоритмов, а также проведение их сравнительного анализа по скоростным характеристикам и эффективности. Для достижения поставленных задач необходимо выполнить следующие пункты:

1) дать сравнительное описание алгоритмов умножения матриц — классического и Винограда;

2) разработать оптимизированную версию алгоритма Винограда;

3) описать три алгоритма умножения матриц схемами алгоритмов;

4) ввести модель вычислений и выполнить в ней оценку трудоемкости алгоритмов;

5) разработать программное обеспечение на языке **C++**;

6) выполнить замеры процессорного времени выполнения реализаций алгоритмов;

7) провести сравнительный анализ алгоритмов по данным результатов замеров.

1 Аналитическая часть

Матрица — это таблица, состоящая из чисел, символов или выражений, расположенных в строках и столбцах. Она служит для представления математического объекта или его свойства. Размер матрицы определяется числом строк и столбцов.

Разработка алгоритмов умножения матриц способствует расширению возможностей обработки больших данных, повышая вычислительную эффективность и обеспечивая более быстрое выполнение сложных математических операций в различных областях науки и технологий.

1.1 Классический алгоритм умножения матриц

Классический метод умножения матриц — это наиболее простой и широко используемый метод умножения двух матриц. Пусть есть две матрицы: A размером $a \times b$ и B размером $b \times c$. Тогда результирующая матрица C будет иметь размерность $a \times c$, и каждый её элемент вычисляется путем сложения скалярных произведений элементов i-ой строки матрицы A и j-ого столбца матрицы B по следующей формуле:

$$C[i][k] = \sum_{k=1}^{b} A[i][k] \cdot B[k][j], \tag{1.1}$$

где $1 \leq i \leq a$ и $1 \leq k \leq c$,
$a[i][k]$ — элемент матрицы A в $i-$ой строке и $k-$ом столбце,
$b[k][j]$ — элемент матрицы B в $k-$ой строке и $j-$ом столбце,
$c[i][k]$ — элемент матрицы C в $i-$ой строке и $k-$ом столбце.

1.2 Метод Винограда умножения матриц

Алгоритм Винограда для умножения матриц — это алгоритм, при котором заранее вычисляются определённые значения, позволяющие сократить количество операций в процессе умножения матриц с помощью тройного цикла. Этот метод особенно эффективен при работе с большими матрицами, поскольку он помогает снизить затраты вычислений, что особенно важно при больших объёмах данных. [1]
Пусть имеется две матрицы A размерности $a \times b$ и B размерности $b \times c$. Тогда **алгоритм Винограда** заключается в следующих по очередности действиях.

1) Создать результирующую матрицу C размером $a \times c$, инициализированную нулями.

2) Создать вспомогательные массивы $rowMult$ и $colMult$ размерностей числа строк матрицы A и столбцов матрицы B соответственно. В ячейке $rowMult[i]$ первого массива будут храниться предварительно вычисленные значения для i-ой строки матрицы A, а в ячейке $colMult[i]$ второго массива — i-ого столбца матрицы B. В j-ой ячейке $rowMult[i][j]$ первого массива будет храниться сумма попарных произведений подряд идущих элементов из $A[i]$, а в j-ой ячейке $colMult[i][j]$ второго массива — сумма попар-

ных произведений подряд идущих элементов из $B[j]$.

3) Используя предварительно вычисленные массивы $rowMult$ и $colMult$, необходимо вычислить каждый элемент результирующей матрицы C по формуле:

$$C[i][j] = Sum[i][j] + div + \sum_{k=1,k=k+2}^{b} (A[i][k] + B[k+1][j]) \cdot (A[i][k+1] + B[k][j]), \quad (1.2)$$

где $Sum[i][j] = -(rowMult[i] + colMult[j])$ и $div = A[i][b-1] \cdot B[b-1][j] \cdot r$ — на случай, если размерность матрицы нечетна, r — остаток от деления b на 2.

1.3 Оптимизированный метод Винограда умножения матриц

Оптимизированный метод Винограда для умножения матриц — улучшенный **классический метод Винограда**, который за счёт некоторых оптимизаций считает результирующую матрицу быстрее, чем оригинал. Для улучшения **метода Винограда** применяются следующие оптимизации.

1) Умножение на 2 заменятеся на двоичный сдвиг целого числа — быстрый побитовый оператор, который изменяет представление числа без выполнения полноценного умножения.

2) Используется предварительное вычисление значений $b-1$, $b/2$, $c/2$, использованных в циклах.

Вывод

Были описаны все алгоритмы и выведены формулы умножения матриц для каждого из них.

2 Конструкторская часть

Подразумевается, что $mat1$ и $mat2$ — это первая и вторая перемножаемая матрица соответственно, $.rows$ и $.cols$ — это строки и столбцы соответствующих матриц.

2.1 Классический алгоритм умножения матриц

Схема алгоритма классического умножения матриц представлена на рисунке 2.1.

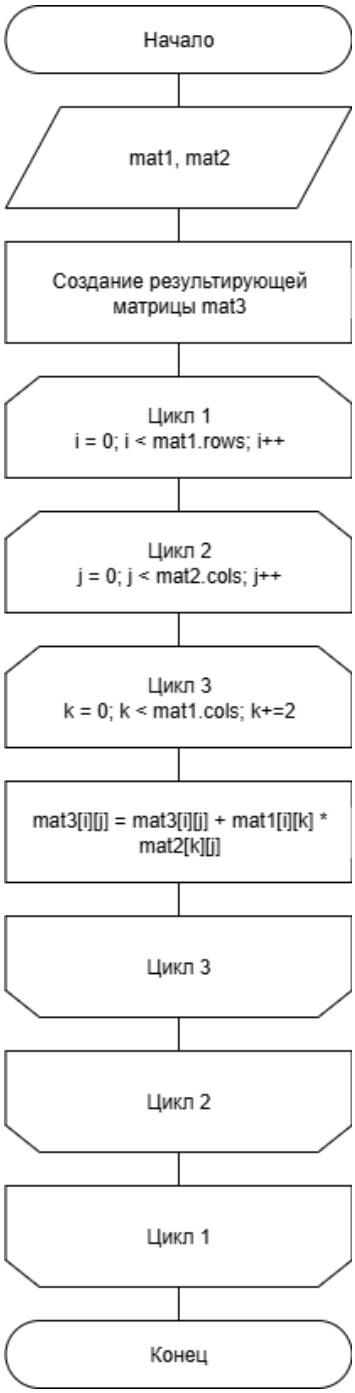

Рисунок 2.1 — Блок-схема **классического алгоритма**

2.2 Алгоритм Винограда умножения матриц

Схема алгоритма Винограда умножения матриц представлена на рисунке 2.2.

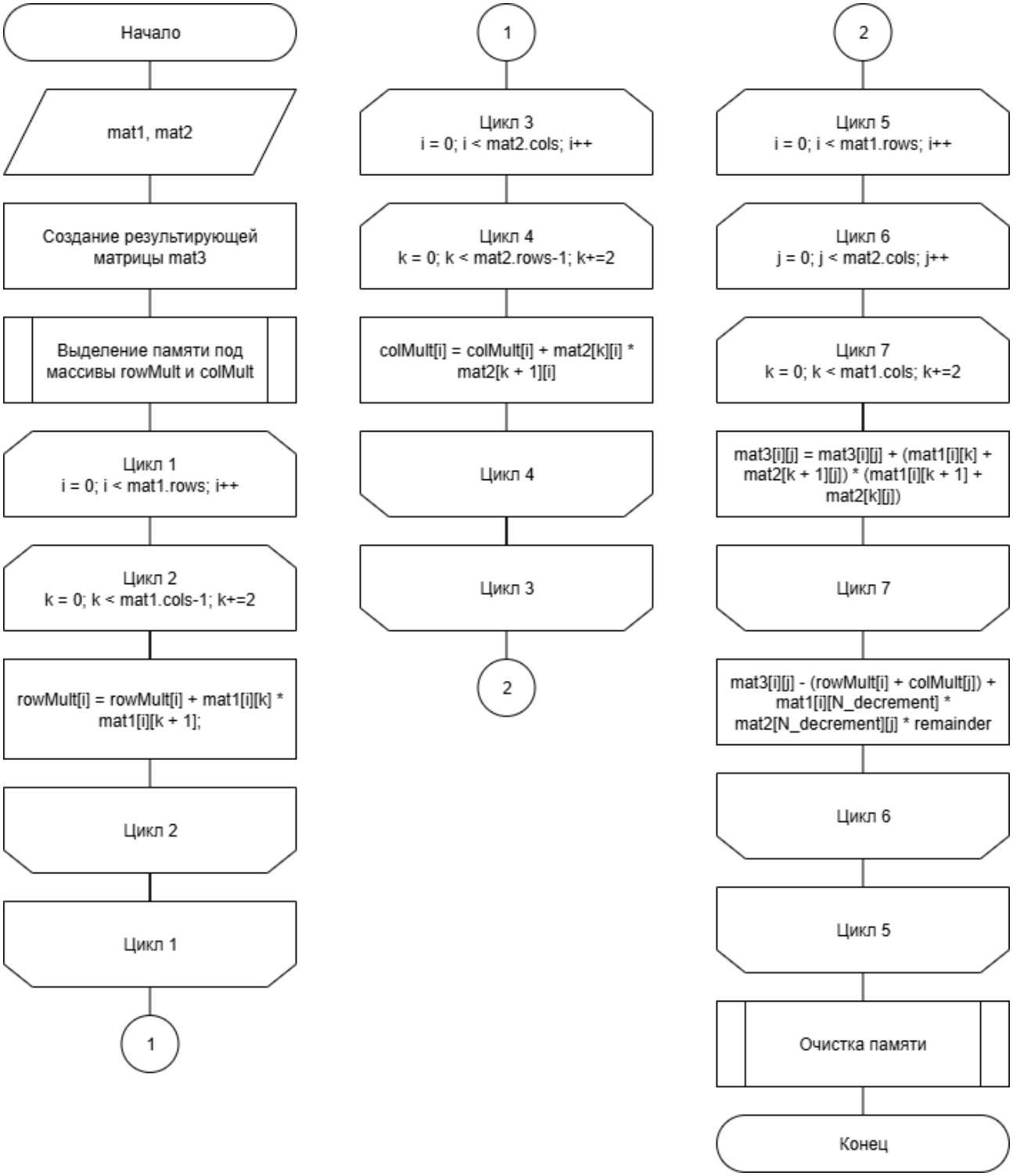

Рисунок 2.2 — Блок-схема **алгоритма Винограда**

2.3 Оптимизированный алгоритм Винограда умножения матриц

Схема оптимизированного алгоритма Винограда умножения матриц представлена на рисунке 2.3.

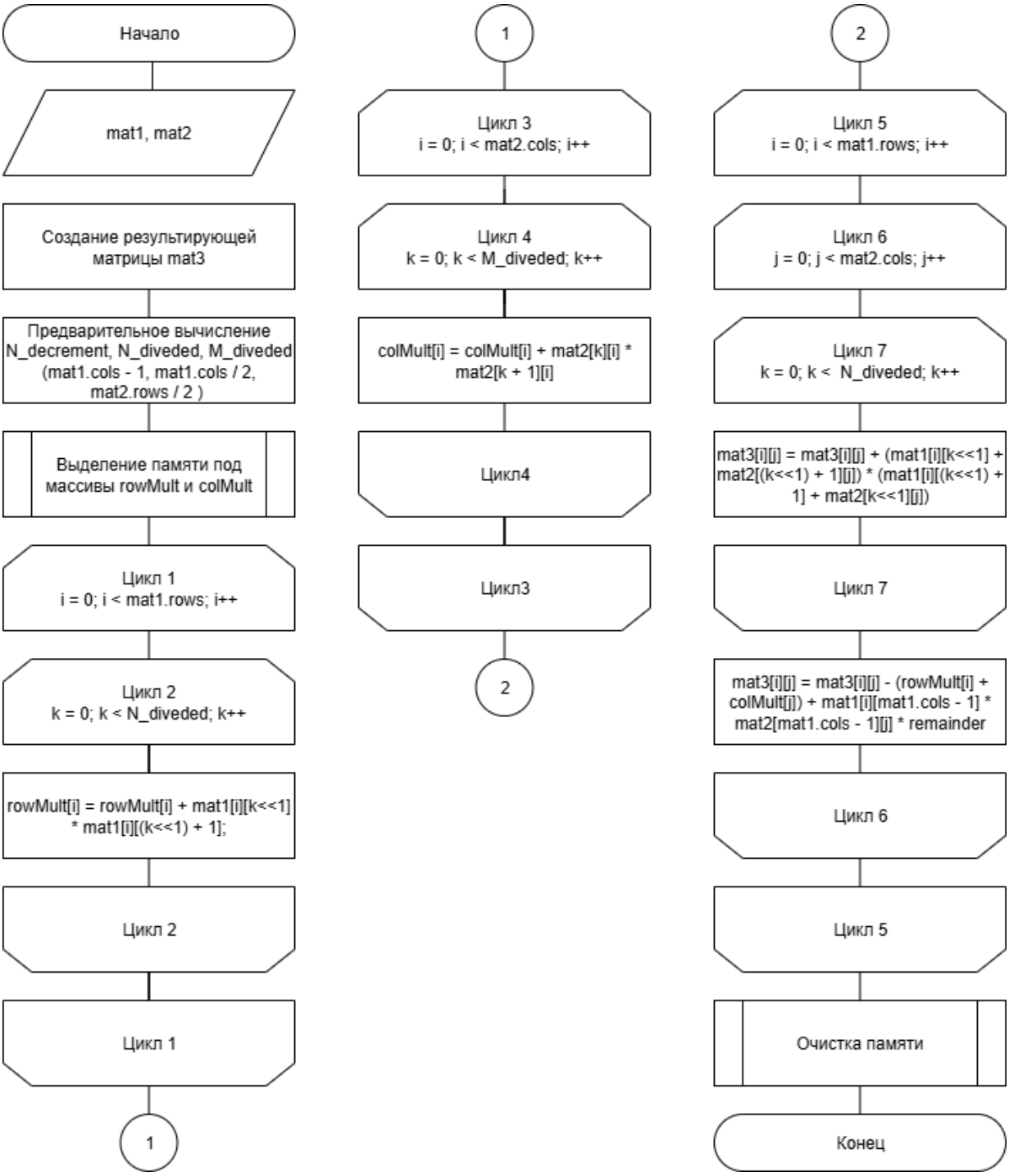

Рисунок 2.3 — Блок-схема **оптимизированного алгоритма Винограда**

Вывод

Была выполнена задача разработки основных алгоритмов.

3 Технологическая часть

3.1 Реализация и выбор средств

Для программной реализации алгоритмов использовалась среда разработки $Visual\ Studio\ Code$ [2]. Язык программирования, на котором была выполнена реализации алгоритмов — $C++$.

В листинге А.1 представлена программная реализация описанных алгоритмов.

3.2 Тестирование программы

Ниже в таблице 3.1 приведено тестирование программы и её проверка на ожидаемый/получаемый результат.

Таблица 3.1 — Результаты тестирования

Описание тестирования	**Входные данные**	**Ожидаемый результат**	**Полученный результат**
Проверка на ввод нечисловых значений	В	Было введено не целочисленное значение. Попробуйте ввести заново	Было введено не целочисленное значение. Попробуйте ввести заново
Проверка на ввод неположительных элементов матрицы	1 -1 1 0 2	Введите 1, чтобы использовать все алгоритмы умножения матриц для ваших данных Введите 2, чтобы протестировать алгоритм произвольными данными Введите количество строк в матрице 1: Введите количество столбцов в матрице 1: Введите количество строк в матрицы 2: Введите количество столбцов в матрице 2: !!!ОШИБКА!!! Размер матрицы должен быть хотя бы 1 на 1	Введите 1, чтобы использовать все алгоритмы умножения матриц для ваших данных Введите 2, чтобы протестировать алгоритм произвольными данными Введите количество строк в матрице 1: Введите количество столбцов в матрице 1: Введите количество строк в матрицы 2: Введите количество столбцов в матрице 2: !!!ОШИБКА!!! Размер матрицы должен быть хотя бы 1 на 1

Таблица 3.1 — Продолжение

Описание тестирования	**Входные данные**	**Ожидаемый результат**	**Полученный результат**
Проверка на несовпадение количества столбцов в матрице 1 и количества строк в матрице 2	1 2 2 1 2	Введите 1, чтобы использовать все алгоритмы умножения матриц для ваших данных Введите 2, чтобы протестировать алгоритм произвольными данными Введите количество строк в матрице 1: Введите количество столбцов в матрице 1: Введите количество строк в матрицы 2: Введите количество столбцов в матрице 2: !!!ОШИБКА!!! Количество столбцов в матрице 1 долнжно равняться количеству строк в матрице 2.	Введите 1, чтобы использовать все алгоритмы умножения матриц для ваших данных Введите 2, чтобы протестировать алгоритм произвольными данными Введите количество строк в матрице 1: Введите количество столбцов в матрице 1: Введите количество строк в матрицы 2: Введите количество столбцов в матрице 2: !!!ОШИБКА!!! Количество столбцов в матрице 1 долнжно равняться количеству строк в матрице 2.

Таблица 3.1 — Продолжение

Описание тестирования	**Входные данные**	**Ожидаемый результат**	**Полученный результат**
Проверка на работу с квадратными матрицами	1 2 2 2 2 1 1 1 1 1 1 1	Введите 1, чтобы использовать все алгоритмы умножения матриц для ваших данных Введите 2, чтобы протестировать алгоритм произвольными данными Введите количество строк в матрице 1: Введите количество столбцов в матрице 1: Введите количество строк в матрицы 2: Введите количество столбцов в матрице 2: (Вывод матрицы 2 на 2 со значениями 2 три раза и времени, потраченного каждым алгоритмом)	Введите 1, чтобы использовать все алгоритмы умножения матриц для ваших данных Введите 2, чтобы протестировать алгоритм произвольными данными Введите количество строк в матрице 1: Введите количество столбцов в матрице 1: Введите количество строк в матрицы 2: Введите количество столбцов в матрице 2: (Вывод матрицы 2 на 2 со значениями 2 три раза и временем, потраченного каждым алгоритмом)
Проверка на работу с прямоугольными матрицами	1 1 3 3 2 1 1 1 1 1 1 1	Введите 1, чтобы использовать все алгоритмы умножения матриц для ваших данных Введите 2, чтобы протестировать алгоритм произвольными данными Введите количество строк в матрице 1: Введите количество столбцов в матрице 1: Введите количество строк в матрицы 2: Введите количество столбцов в матрице 2: (Вывод матрицы 1 на 2 со значениями 3 три раза и времени, потраченного каждым алгоритмом)	Введите 1, чтобы использовать все алгоритмы умножения матриц для ваших данных Введите 2, чтобы протестировать алгоритм произвольными данными Введите количество строк в матрице 1: Введите количество столбцов в матрице 1: Введите количество строк в матрицы 2: Введите количество столбцов в матрице 2: (Вывод матрицы 1 на 2 со значениями 3 три раза и временем, потраченного каждым алгоритмом)

Вывод

Было проведено тестирование программы на работоспсобность.

4 Исследовательская часть

4.1 Сравнение времени выполнения программы

Исследование по измерению времени работы реализаций алгоритмов проводилось на ноутбуке Huawei Matebook d16 [3]. Характеристики ноутбука: 64-разрядная операционная система Windows 11 [4], 8-ядерный процессор Intel Core i5-12450H [5] x64 с тактовой частотой 2 Ггц, оперативная память DDR4 8 ГБ. Для компиляции кода использовался компилятор g++.

На рисунке 4.1 представлены графики зависимости времени работы реализаций всех вышеописанных алгоритмов (**классического**, **Винограда** и **оптимизированного Винограда**) от размера умножаемых матриц. На вход реализации алгоритмы получали квадратные матрицы размером $n \times n, n \in [10; 1310]$ с шагом, равным 100. Все матрицы заполнялись случайными данными в формате int. Время выполнения реализации алгоритма для четных и нечетных n не будет изменяться в силу отсутствия условного оператора. Все матрицы заполнялись случайными целочисленными данными. Использованные данные для графиков — это результат измерения с помощью функции $modeTest$ в программе.

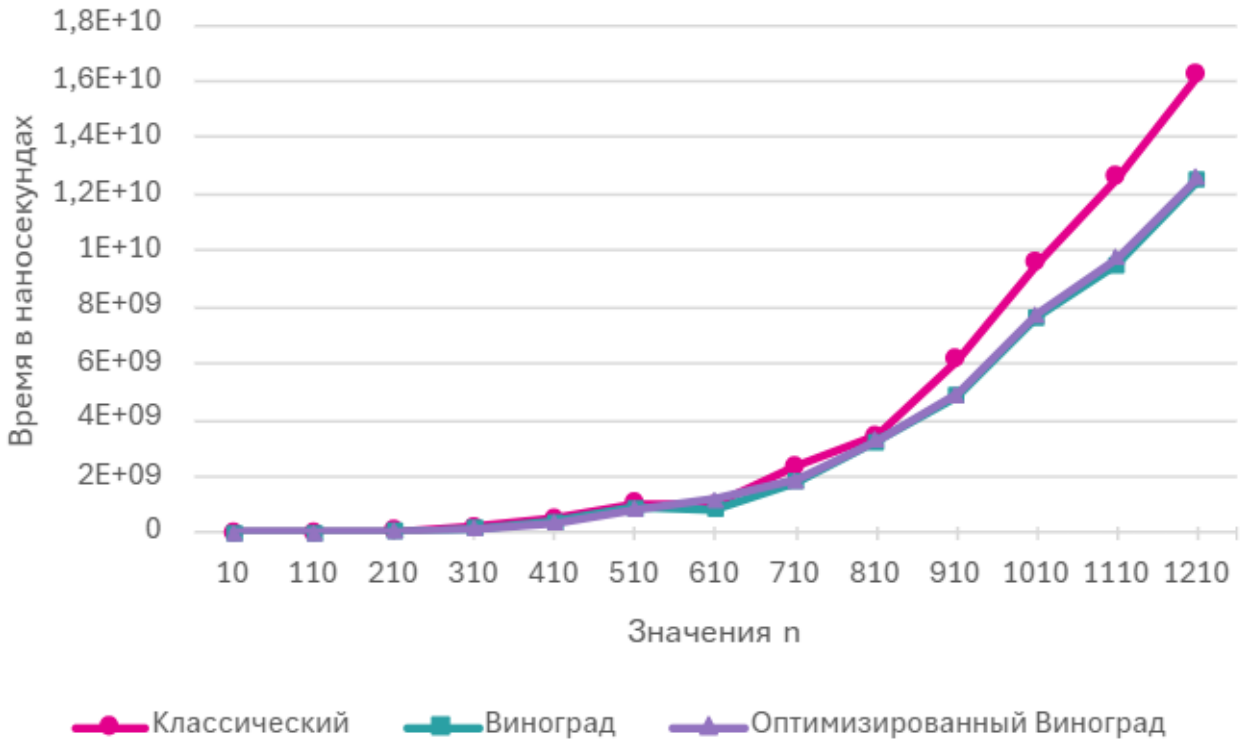

Рисунок 4.1 — Графики времени работы реализаций алгоритмов в зависимости от размеров матриц

4.2 Оценка трудоемкости алгоритмов

Ниже приведен теоретический расчёт сложности всех алгоритмов. Модель трудоемкости введена в таблице 4.1.

Таблица 4.1 — Модель трудоемкости

Название базовой операции	**Стоимость**
Сложение	1
Вычитание	1
Присвивание	1
Равно	1
Меньше	1
Больше	1
Меньше или равно	1
Больше или равно	1
Не равно	1
Инкремент	1
Декремент	1
Плюс с присваиванием	1
Индексирование	1
Умножение	2
Деление	2
Целочисленное деление	2
Остаток от деления	2

Оценка трудоемкости цикла for от 0 до M имеет вид:

$$f_{\text{цикла}} = f_{\text{иниц}} + f_{\text{усл}} + M \cdot (f_{\text{тело}} + f_{\text{инк}} + f_{\text{усл}}), \tag{4.1}$$

где $f_{\text{иниц}}$ — стоимость инициализации переменной цикла,
$f_{\text{усл}}$ — стоимость условия, накладываемого на переменную,
$f_{\text{тел}}$ — стоимость тела цикла,
$f_{\text{инк}}$ — стоимость операции увеличения переменной цикла.

Пусть первая и вторая матрицы имеют размерности $A \times B$ и $B \times C$ соответственно.

Классический алгоритм (1.1):

$$2 + A \cdot (2 + 2 + C \cdot 2 + 3 + 2 + B \cdot (2 + 9)) = 2 + 4 \cdot A + 7 \cdot A \cdot C + 11 \cdot A \cdot B \cdot C \tag{4.2}$$

Алгоритм Винограда (1.2):

Ниже приведены рассчеты стоимости каждого цикла.

1. Первый цикл:

$$15/2 \cdot A \cdot B + 5 \cdot A + 2 \tag{4.3}$$

2. Второй цикл:

$$15/2 \cdot B \cdot C + 5 \cdot B + 2 \tag{4.4}$$

3. Третий цикл:

$$17/2 \cdot A \cdot B \cdot C + 13 \cdot A \cdot C + 4 \cdot A + 2 \tag{4.5}$$

В лучшем случае B будет кратно 2 и алгоритм будет иметь следующую трудоемкость:

$$8 + 9 \cdot A + 5 \cdot B + \frac{15}{2} \cdot A \cdot B + \frac{15}{2} \cdot B \cdot C + 13 \cdot A \cdot C + 13 \cdot A \cdot B \cdot C \tag{4.6}$$

В худшем случае B не будет кратно 2 и алгоритм будет иметь следующую трудоемкость:

$$8 + 13 \cdot A + 5 \cdot B + \frac{47}{2} \cdot A \cdot B + \frac{15}{2} \cdot B \cdot C + 13 \cdot A \cdot C + 13 \cdot A \cdot B \cdot C \tag{4.7}$$

4.3 Вывод

Из расчётов следует, что при работе с матрицами большого размера алгоритм Винограда быстрее по сравнению с классическим, так как коэффициент при последнем множителе меньше (17/2 < 11). Наиболее быстрым является алгоритм Винограда.

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы была достигнута цель работы: реализация алгоритмов, а также сравнительный анализ алгоритмов умножения матриц.

Также были выполнены все задачи:

1) дано сравнительное описание алгоритмов умножения матриц — классического и Винограда;

2) разработана оптимизированная версия алгоритма Винограда;

3) описаны три алгоритма умножения матриц схемами алгоритмов;

4) ввеедена модель вычислений и в ней выполнена оценка трудоемкости алгоритмов;

5) разработано программное обеспечение на языке **C++**;

6) выполнины замеры процессорного времени выполнения реализаций алгоритмов;

7) проведен сравнительный анализ алгоритмов по данным результатов замеров.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы современных алгоритмов 2-е дополненное издание / Дж. Макконелл // под ред. С.Кландо, доп. М.В.Ульянова. — Москва: ТЕХНОСФЕРА, 2004. — 370 с.

2. Visual Studio Code documentation. [Электронный ресурс]. URL: `https://code.visualstudio.com/docs` (дата обращения: 3 октября 2025).

3. Huawei. [Электронный ресурс]. URL: `https://consumer.huawei.com/ru/laptops/matebook-14-2024/specs/?ysclid=mg5k19xzko358223160` (дата обращения: 4 октября 2025).

4. Microsoft Ignite. [Электронный ресурс]. URL: `https://learn.microsoft.com/ru-ru/windows` (дата обращения: 4 октября 2025).

5. NanoReview. [Электронный ресурс]. URL: `https://nanoreview.net/ru/cpu/intel-core-i5-12450h` (дата обращения: 4 октября 2025).

ПРИЛОЖЕНИЕ А

В листинге А.1 представлена программная реализация описанных алгоритмов.

```
#include <iostream>
#include <chrono>
#include <cmath>

using namespace std;
using namespace chrono;

const int rnd_a = 16385;
const int rnd_c = 1;
const int rnd_m = 65536;

bool mode_type = true;

struct incorrect_size_error {
  incorrect_size_error(string exceptionMessage) { cerr << "ОШИБКА!!!!!! " <<
    exceptionMessage << endl; }
};

unsigned get_random_value() {
  static unsigned seed = 1;
  seed = (seed * rnd_a + rnd_c) % rnd_m;
  return seed;
}

int safe_input() {

  int digit;

  while (true) {

    int temp; cin >> temp;
    if (cin.fail()) {
      cin.clear();
      cout << "Было введено не целочисленное значение. Попробуйте ввести заново." <<
    endl;
      cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    else {
      digit = temp;
      break;
    }

  }

```

```
  return digit;

}

struct Timer {

  Timer() {
    start = high_resolution_clock::now();
    end = high_resolution_clock::now();
  }

  void start_time() { start = high_resolution_clock::now(); }
  void stop_time() { end = high_resolution_clock::now(); }

  void get_time() {
    auto duration = duration_cast<nanoseconds>(end - start);
    cout << "Время работы алгоритма составило " << duration.count() << "
   наносекунд" << endl;
  }

  private:
  time_point<high_resolution_clock> start;
  time_point<high_resolution_clock> end;
};

struct Matrix {

  int cols, rows;
  int** values;

  Matrix(int r = 0, int c = 0) : rows(r), cols(c) {
    values = new int*[rows];
    for (int i = 0; i < rows; i++)
    values[i] = new int[cols] {};
  }

  ~Matrix() {
    for (int i = 0; i < rows; i++)
    delete[] values[i];
    delete[] values;
  }

  void enter() {
    for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
    values[i][j] = safe_input();
  }
```

```

  void fill_random_values() {
    for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
    values[i][j] = get_random_value();
  }

  void print() const {
    for (int i = 0; i < rows; i++) {
      for (int j = 0; j < cols; j++)
      cout << values[i][j] << " ";
      cout << endl;
    }
  }

  int* operator[](int index) { return values[index]; }
  int* operator[](int index) const { return values[index]; }

};

void classic_multiply_algorithm(Matrix& mat1, Matrix& mat2) {

  cout << "~~~ Классический алгоритм ~~~" << endl;

  Matrix mat3(mat1.rows, mat2.cols);
  Timer time;

  time.start_time();
  for (int i = 0; i < mat1.rows; i++)
  for (int j = 0; j < mat2.cols; j++)
  for (int k = 0; k < mat1.cols; k++)
  mat3[i][j] += (mat1[i][k] * mat2[k][j]);

  time.stop_time();
  time.get_time();

  if (mode_type)
  mat3.print();

}

void winograd_algorithm(Matrix& mat1, Matrix& mat2) {

  cout << "~~~ Алгоритм Винограда ~~~" << endl;

  Matrix mat3(mat1.rows, mat2.cols);
  int remainder = mat1.cols % 2;
```

```
  int* rowMult = new int[mat1.rows]{0};
  int* colMult = new int[mat2.cols]{0};
  Timer time;

  time.start_time();
  for (int i = 0; i < mat1.rows; i++)
  for (int k = 0; k < mat1.cols - 1; k+=2)
  rowMult[i] += mat1[i][k] * mat1[i][k + 1];

  for (int i = 0; i < mat2.cols; i++)
  for (int k = 0; k < mat2.rows - 1; k+=2)
  colMult[i] += mat2[k][i] * mat2[k + 1][i];

  for (int i = 0; i < mat1.rows; i++) {
    for (int j = 0; j < mat2.cols; j++) {
      for (int k = 0; k < mat1.cols - 1; k+=2)
      mat3[i][j] += ((mat1[i][k] + mat2[k + 1][j]) * (mat1[i][k + 1] + mat2[
    k][j]));
      mat3[i][j] = mat3[i][j] - (rowMult[i] + colMult[j]) + mat1[i][mat1.
    cols - 1] * mat2[mat1.cols - 1][j] * remainder; } }

  time.stop_time();
  time.get_time();

  if (mode_type)
  mat3.print();

  delete[] rowMult;
  delete[] colMult;

}

void winograd_optimized_algorithm(Matrix& mat1, Matrix& mat2) {

  cout << "~~~ Оптимизированный алгоритм Винограда ~~~" << endl;

  Matrix mat3(mat1.rows, mat2.cols);
  int remainder = mat1.cols % 2;
  int* rowMult = new int[mat1.rows]{0};
  int* colMult = new int[mat2.cols]{0};
  int N_decrement = mat1.cols - 1;
  int N_diveded = mat1.cols / 2;
  int M_diveded = mat2.rows / 2;
  Timer time;

  time.start_time();
  for (int i = 0; i < mat1.rows; i++)
```

```
  for (int k = 0; k < N_diveded; k++)
  rowMult[i] += mat1[i][k<<1] * mat1[i][(k<<1) + 1];

  for (int i = 0; i < mat2.cols; i++)
  for (int k = 0; k < M_diveded; k++)
  colMult[i] += mat2[k<<1][i] * mat2[(k<<1) + 1][i];

  for (int i = 0; i < mat1.rows; i++) {
    for (int j = 0; j < mat2.cols; j++) {
      for (int k = 0; k < N_diveded; k++)
      mat3[i][j] += ((mat1[i][k<<1] + mat2[(k<<1) + 1][j]) * (mat1[i][(k<<1) + 1] + mat2[k<<1][j]));
      mat3[i][j] = mat3[i][j] - (rowMult[i] + colMult[j]) + mat1[i][N_decrement] * mat2[N_decrement][j] * remainder; } }

  time.stop_time();
  time.get_time();

  if (mode_type)
  mat3.print();

  delete[] rowMult;
  delete[] colMult;

}

void matricies_size_reader(int& m1_rows, int& m1_cols, int& m2_rows, int& m2_cols) {

  cout << "Введите количество строк в матрице 1" << endl;
  m1_rows = safe_input();
  cout << "Введите количество столбцов в матрице 1" << endl;
  m1_cols = safe_input();
  cout << "Введите количество строк в матрице 2" << endl;
  m2_rows = safe_input();
  cout << "Введите количество столбцов в матрице 2" << endl;
  m2_cols = safe_input();

  if ((m1_rows < 1) || (m1_cols < 1) || (m2_rows < 1) || (m2_cols < 1))
  throw incorrect_size_error("Размер матрицы должен быть хотя бы 1 на 1");
  if (m1_cols != m2_rows)
  throw incorrect_size_error("Количество столбцов в матрице 1 долнжно равняться количеству строк в матрице 2");

}

void mode_test() {
```

```
  cout << "~~~ Режим тестирования для матрицы n на n ~~~" << endl;

  for (int n = 10; n < 1300; n+=100) {

    Matrix mat(n, n);
    mat.fill_random_values();

    cout << "~~~~~ результаты для n = " << n << " ~~~~~" << endl;

    classic_multiply_algorithm(mat, mat);
    winograd_algorithm(mat, mat);
    winograd_optimized_algorithm(mat, mat);

  }

}

void mode_user() {

  int m1_rows, m1_cols, m2_rows, m2_cols;

  try { matricies_size_reader(m1_rows, m1_cols, m2_rows, m2_cols); }
  catch (incorrect_size_error) { return; }

  Matrix mat1(m1_rows, m1_cols), mat2(m2_rows, m2_cols);

  cout << "Введите элементы 1:" << endl;
  mat1.enter();

  cout << "Введите элементы 2:" << endl;
  mat2.enter();

  classic_multiply_algorithm(mat1, mat2);
  winograd_algorithm(mat1, mat2);
  winograd_optimized_algorithm(mat1, mat2);

}

int main() {

  int menu;

  while (true) {

    cout << "Меню:" << endl << "Введите 1, чтобы использовать все алгоритмы
```

```
    умножения матриц для ваших данных"
     << endl << "Введите 2, чтобы протестировать алгоритм произвольными данными" <<
    endl;

     menu = safe_input();

     if (menu == 1) { mode_user(); break; }
     else if (menu == 2) { mode_type = false; mode_test(); break; }
     else { cout << "Введите 1 или 2" << endl; }

   }

   return 0;

}
```

Листинг А.1 — Программная реализация разных алгоритмов умножения матриц